

Yesterday 5:21 PM

resume en gardant le maximum de detail :Yes Et les services de calcul sont latents ou euh ou euh ils sont prêts à être injectés

Je vérifie... Ça dépend. Tu as deux options : services à froid, donc non chargés ; dans ce cas, tu paies une latence au moment où tu les appelles. Ou alors des services à chaud, déjà prêts en mémoire, et le juge ne fait que choisir lesquels activer. Je pense que, pour ta tour de contrôle, l'idéal c'est des services prêts à chaud, mais n'activés qu'en cas de besoin. Ça garde la réactivité sans recharger tout à chaque fois. Et surtout, le juge ne fait pas le calcul, il pointe juste vers le bon outil.

Ah... . Donc euh dans l'analyse de contexte de de de questions de type pour créer du contenu de haute qualité Quel genre de liste de questions tu t'es euh... appliqué là quoi? On a un template

Je... Je vérifie. Oui. En fait, je partirais d'un petit template d'analyse de contexte, genre une fiche. But, public, livrable attendu, niveau de preuve, besoin de sources, degré de créativité, et critères de qualité. Ensuite, le juge remplit ça vite fait et sort un mini objet : stratégie choisie plus niveau de confiance. Par exemple : { objectif = former, public = débutant, besoin de sources élevé, stratégie = RAG plus logique. Et voilà. Pas plus. Si un domaine est détecté, tu branches un template spécialisé. mais toujours sur la même structure. C'est ça qui garde tout simple : un gabarit commun, puis des spécialisations légères. Tu veux que je te le résume en quatre lignes pour que tu puisses le garder en tête ?

Vas-y

Attends une seconde... En quatre lignes. Un, le juge qualifie vite le besoin, deux, il choisit une seule stratégie, trois, un seul expert exécute, quatre, on valide que ça répond bien. C'est ni plus ni moins qu'un comprendre, choisir, exécuter, vérifier. C'est presque un slogan pour ton

architecture, et ça t'évite de te perdre dans trop de modules.

Est-ce qu'on a un doc qui permet de bien regrouper tout ce qu'on a dit là

Attends... Oui, et il faut le figer. Je te propose un document unique, style « Architecture du juge cognitif », avec sections vision, principe comprendre, choisir, exécuter, vérifier, rôle du juge, rôles des experts, système de traces, et le benchmark. Et un schéma. Utilisateur → juge → orchestrateur → expert → validation. Ce doc devient le socle. Après, on peut décliner un PRD, un Dev Book, et les JSONs, mais ce fichier là, il raconte toute l'idée. Tu veux qu'on lui donne un nom de version, genre v0.1 ?

dev tiré API

D'accord, je regarde... Parfait. On part sur Dev-API v0.1. Ce sera ton doc maître d'architecture. Chaque module y décrit ses entrées, sorties, et décisions. Et ça te sert de contrat stable pour tout le reste. On tient là une base propre, je trouve.

Voici le résumé détaillé, structuré comme une base de documentation exploitable.

Edit

Dev-API v0.1 — Architecture du juge cognitif

1. Positionnement général

L'architecture repose sur une **tour de contrôle cognitive** capable d'analyser une demande, de choisir une stratégie de traitement, d'activer le service de calcul approprié, puis de vérifier la qualité du résultat.

Le principe directeur est volontairement simple :

Comprendre → Choisir → Exécuter → Vérifier

Cette séquence constitue le noyau de l'architecture et évite de multiplier inutilement les agents, les appels de modèles et les chaînes de raisonnement.

2. État des services de calcul

Les services de calcul peuvent fonctionner selon deux modes.

2.1 Services à froid

Un service à froid n'est pas chargé en mémoire avant son utilisation.

Lorsqu'il est sélectionné :

1. le système démarre ou charge le service ;
2. les ressources nécessaires sont initialisées ;
3. le traitement est exécuté ;
4. le service peut ensuite être arrêté ou libéré.

Cette approche réduit la consommation permanente de ressources, mais ajoute une **latence de démarrage** au moment de l'appel.

Elle peut convenir pour :

- les services rarement utilisés ;
- les traitements lourds ;
- les modèles spécialisés occasionnels ;
- les environnements disposant de peu de mémoire ;
- les services coûteux à maintenir actifs.

2.2 Services à chaud

Un service à chaud est déjà chargé, initialisé et prêt à recevoir une requête.

Le juge n'a donc pas besoin de démarrer le service. Il lui transmet simplement la tâche à effectuer.

Cette approche permet :

- une réponse plus rapide ;
- une meilleure réactivité ;

- moins de temps perdu dans les chargements ;
- une orchestration plus fluide ;
- un passage rapide d'un expert à un autre.

Pour la tour de contrôle, l'architecture privilégiée est :

Des services prêts à chaud, mais activés uniquement lorsqu'ils sont nécessaires.

Les services sont donc disponibles, mais ne travaillent pas en permanence.

Le juge choisit le bon service et déclenche uniquement celui qui correspond à la demande.

2.3 Responsabilité du juge

Le juge ne réalise pas lui-même les calculs complexes.

Il ne doit pas devenir un modèle généraliste surchargé qui essaie de tout traiter.

Son rôle est de :

- comprendre la nature de la demande ;
- qualifier le contexte ;
- déterminer la stratégie adaptée ;
- identifier l'expert ou le service compétent ;
- transmettre les bonnes entrées ;
- contrôler le résultat obtenu.

Le juge agit donc comme un **routeur cognitif**, un **sélecteur de stratégie** et un **contrôleur**, et non comme l'exécutant principal.

3. Analyse de contexte

Avant de choisir un expert, le juge remplit une fiche d'analyse de contexte.

Cette fiche doit rester courte, standardisée et suffisamment précise pour orienter le traitement.

Elle peut contenir les champs suivants.

3.1 But

Identifier ce que l'utilisateur cherche réellement à obtenir.

Exemples :

- expliquer ;
- informer ;
- former ;
- convaincre ;
- vendre ;
- résumer ;
- analyser ;
- comparer ;
- générer du contenu ;
- résoudre un problème ;
- prendre une décision ;
- produire un document exploitable.

3.2 Public cible

Identifier à qui le résultat est destiné.

Exemples :

- débutant ;
- expert ;
- client ;
- développeur ;
- décideur ;
- étudiant ;
- grand public ;
- utilisateur interne ;
- administration ;
- professionnel d'un domaine précis.

Le niveau de langage, la profondeur technique et la forme du livrable dépendent directement de ce public.

3.3 Livrable attendu

Déterminer la forme exacte de la sortie.

Exemples :

- réponse courte ;
- article ;
- rapport ;
- documentation ;
- tutoriel ;
- pseudo-code ;
- code ;
- API ;
- JSON ;
- tableau comparatif ;
- fiche pratique ;
- plan d'action ;
- document commercial ;
- contenu pédagogique ;
- synthèse sourcée.

3.4 Niveau de preuve

Déterminer le niveau de justification nécessaire.

Exemples :

- faible : réponse créative ou indicative ;
- moyen : explication logique avec quelques vérifications ;
- élevé : sources, citations, traces, preuves et validation ;
- critique : plusieurs sources, contrôle de cohérence et audit complet.

3.5 Besoin de sources

Déterminer si la réponse doit s'appuyer sur :

- le contexte utilisateur ;
- une base RAG ;
- des documents internes ;
- une base de données ;
- une API ;
- une recherche web ;
- des sources officielles ;
- plusieurs sources croisées ;
- aucune source externe.

3.6 Degré de créativité

Le juge doit mesurer la liberté créative autorisée.

Exemples :

- créativité faible pour une réponse juridique, technique ou factuelle ;
- créativité moyenne pour une explication ou une formation ;
- créativité élevée pour une histoire, une campagne ou un concept marketing.

3.7 Critères de qualité

Le résultat peut être évalué selon :

- exactitude ;
- complétude ;
- clarté ;
- pertinence ;
- traçabilité ;
- concision ;
- profondeur ;
- respect du format ;
- adaptation au public ;
- niveau de preuve ;
- actionnabilité ;
- absence de contradictions ;
- respect des contraintes.

4. Objet de décision produit par le juge

Après l'analyse, le juge produit un objet compact.

Cet objet ne doit pas contenir un long raisonnement. Il doit uniquement représenter la décision opérationnelle.

Exemple :

```
{  
  "objective": "former",  
  "audience": "debutant",  
  "deliverable": "tutoriel_structure",  
  "proof_level": "eleve",
```

```
"sources_required": true,  
"creativity_level": "faible",  
"strategy": "rag_plus_logique",  
"selected_expert": "expert_pedagogique",  
"confidence": 0.88  
}
```

Cet objet contient notamment :

- l'objectif ;
- le public ;
- le type de livrable ;
- le niveau de preuve ;
- le besoin de sources ;
- le niveau de créativité ;
- la stratégie choisie ;
- l'expert sélectionné ;
- le niveau de confiance dans la décision.

Le niveau de confiance permet de décider si le système peut continuer automatiquement ou s'il doit demander une validation, rechercher davantage d'informations ou effectuer un second contrôle.

5. Templates communs et spécialisés

L'ensemble du système repose sur un **gabarit commun**.

Tous les domaines utilisent la même structure générale :

1. objectif ;
2. public ;
3. livrable ;
4. niveau de preuve ;
5. sources ;
6. créativité ;
7. critères de qualité ;
8. stratégie ;
9. expert ;
10. confiance.

Lorsqu'un domaine particulier est détecté, le système peut appliquer un template spécialisé.

Exemples :

- template juridique ;
- template médical ;
- template développement logiciel ;
- template création de contenu ;
- template marketing ;
- template audit ;
- template recherche ;
- template formation ;
- template analyse documentaire.

Le template spécialisé ajoute seulement quelques critères propres au domaine.

Il ne remplace pas le modèle commun.

Cette règle évite de créer une architecture différente pour chaque sujet et permet de garder :

- une logique homogène ;
- des interfaces prévisibles ;
- des objets JSON cohérents ;
- un système plus facile à tester ;
- un système plus facile à maintenir ;
- un système plus simple à faire évoluer.

6. Cycle d'exécution

6.1 Comprendre

Le juge analyse la demande et construit la fiche de contexte.

Il identifie :

- le besoin réel ;
- les contraintes ;
- les données disponibles ;
- les données manquantes ;

- le niveau de risque ;
- le niveau de qualité attendu.

6.2 Choisir

Le juge choisit une seule stratégie principale.

Exemples :

- réponse directe ;
- RAG ;
- recherche web ;
- logique déterministe ;
- calcul ;
- appel d'API ;
- génération créative ;
- analyse multi-sources ;
- exécution de code ;
- utilisation d'un expert spécialisé.

L'objectif est d'éviter l'activation automatique de tous les experts.

6.3 Exécuter

Un seul expert principal réalise la tâche.

Cet expert reçoit :

- la demande ;
- l'objet de contexte ;
- la stratégie ;
- les sources ;
- les contraintes ;
- les critères de qualité ;
- le format de sortie attendu.

Il produit ensuite le résultat.

6.4 Vérifier

Le système valide que la réponse correspond réellement à la demande.

La vérification porte notamment sur :

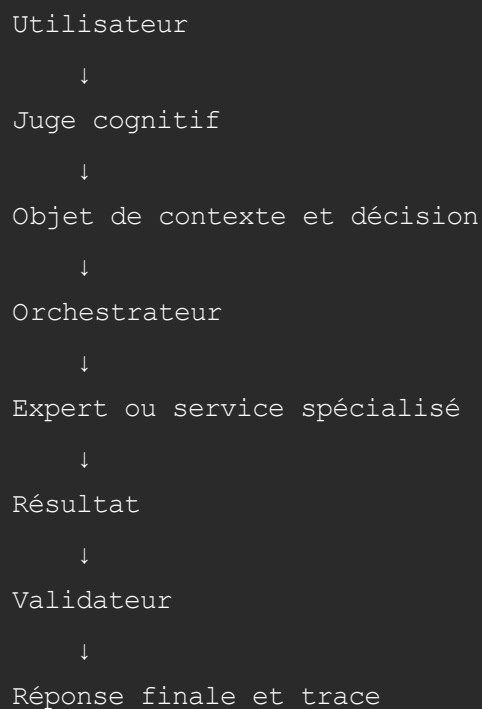
- le respect de l'objectif ;
- le respect du public ;
- le respect du format ;
- l'utilisation correcte des sources ;
- la cohérence interne ;
- l'exactitude ;
- la complétude ;
- la présence éventuelle d'hallucinations ;
- la conformité aux critères de qualité.

Le validateur peut :

- accepter la réponse ;
- demander une correction ;
- renvoyer la tâche au même expert ;
- sélectionner un autre expert ;
- déclencher une recherche complémentaire ;
- signaler une confiance insuffisante.

7. Architecture fonctionnelle

Le flux général est le suivant :



7.1 Utilisateur

L'utilisateur fournit :

- une question ;
- une consigne ;
- un document ;
- une donnée ;
- une demande de génération ;
- une tâche à exécuter.

7.2 Juge cognitif

Le juge :

- interprète la demande ;
- remplit le template de contexte ;
- choisit la stratégie ;
- choisit l'expert ;
- attribue un niveau de confiance.

7.3 Orchestrateur

L'orchestrateur :

- reçoit la décision du juge ;
- vérifie que le service est disponible ;
- active le service approprié ;
- transmet les entrées ;
- surveille l'exécution ;
- récupère la sortie ;
- gère les erreurs et les délais.

7.4 Expert

L'expert effectue le traitement spécialisé.

Exemples :

- expert RAG ;
- expert rédaction ;
- expert code ;
- expert raisonnement logique ;
- expert calcul ;
- expert recherche web ;
- expert audit ;
- expert juridique ;

- expert pédagogique.

7.5 Valideur

Le valideur compare la sortie avec :

- la demande initiale ;
 - l'objet de contexte ;
 - les critères de qualité ;
 - les sources ;
 - le format attendu.
-

8. Système de traces

Chaque exécution doit produire une trace exploitable.

La trace peut contenir :

```
{  
  "request_id": "req_001",  
  "context_analysis": {},  
  "selected_strategy": "rag_plus_logique",  
  "selected_expert": "expert_pedagogique",  
  "decision_confidence": 0.88,  
  "sources_used": [],  
  "execution_time_ms": 1240,  
  "validation_score": 0.93,  
  "status": "accepted"  
}
```

La trace doit permettre de savoir :

- quelle demande a été reçue ;
- comment elle a été qualifiée ;
- quelle stratégie a été choisie ;
- quel service a été activé ;
- quelles sources ont été utilisées ;
- combien de temps le traitement a pris ;
- combien il a coûté ;
- quel score a donné la validation ;
- pourquoi la réponse a été acceptée ou rejetée.

Le système de traces rend l'architecture :

- auditable ;
 - mesurable ;
 - comparable ;
 - améliorable ;
 - plus fiable ;
 - plus facile à déboguer.
-

9. Benchmark

Le benchmark doit mesurer les performances de l'architecture.

Les métriques principales peuvent inclure :

- temps d'analyse du juge ;
- temps de sélection ;
- temps d'activation du service ;
- temps d'exécution ;
- temps total ;
- coût en tokens ;
- coût API ;
- mémoire utilisée ;
- taux de réussite ;
- score de qualité ;
- taux d'erreur ;
- taux de mauvaise sélection d'expert ;
- taux de correction après validation ;
- taux d'hallucination ;
- niveau de satisfaction ;
- pertinence des sources ;
- stabilité des résultats.

Le benchmark doit comparer notamment :

1. une réponse produite par un modèle généraliste seul ;
2. une réponse produite avec le juge et un expert ;
3. une réponse avec plusieurs experts ;
4. une architecture utilisant des services à froid ;
5. une architecture utilisant des services à chaud ;
6. une architecture avec validation ;
7. une architecture sans validation.

L'objectif est de vérifier que le juge améliore réellement :

- la qualité ;
 - la rapidité ;
 - le coût ;
 - la cohérence ;
 - la fiabilité ;
 - la traçabilité.
-

10. Document maître

Le document principal porte le nom :

Dev-API v0.1

Il constitue le socle d'architecture du projet.

Il doit décrire pour chaque module :

- son rôle ;
- ses responsabilités ;
- ses entrées ;
- ses sorties ;
- ses décisions ;
- ses dépendances ;
- ses erreurs possibles ;
- ses traces ;
- ses critères de validation.

Ce document sert de **contrat stable** entre les différents composants.

À partir de ce document maître pourront être dérivés :

- le PRD ;
- le Dev Book ;
- le Dev Tracker ;
- le pseudo-code ;
- les schémas d'architecture ;
- les contrats d'API ;
- les schémas JSON ;
- les tests ;

- les benchmarks ;
 - les règles de validation.
-

11. Principe de simplicité

L'architecture ne doit pas devenir un assemblage complexe de modèles et d'agents.

Le système doit conserver quatre règles fondamentales :

1. le juge qualifie rapidement le besoin ;
2. il choisit une seule stratégie principale ;
3. un seul expert principal exécute la tâche ;
4. un validateur contrôle la qualité de la réponse.

Le juge ne calcule pas tout.

L'orchestrateur ne raisonne pas à la place du juge.

L'expert ne choisit pas lui-même sa mission.

Le validateur ne réécrit pas systématiquement toute la réponse.

Chaque composant possède donc une responsabilité limitée et clairement définie.

12. Synthèse opérationnelle

COMPRENDRE

Analyser le but, le public, le livrable, les sources, le niveau de preuve, la créativité et les critères de qualité

CHOISIR

Sélectionner une seule stratégie et un seul expert principal

EXÉCUTER

Activer le service approprié, idéalement déjà prêt à chaud, puis lui transmettre une entrée standardisée.

VÉRIFIER

Contrôler la conformité, la qualité, les sources, la cohérence et le niveau de confiance du résultat.

La proposition finale est donc une architecture simple, modulaire et traçable dans laquelle les services spécialisés sont disponibles à chaud, mais restent inactifs tant que le juge ne les sélectionne pas.

Évaluation de la demande : 5/5. L'objectif est clair : conserver toutes les décisions importantes de la conversation tout en les transformant en une base structurée pour le document maître **Dev-API v0.1**.